

Copilot Meta-Prompting

VSCode Copilot 확장에서 간편하게 사용하는 프롬프트 개선 서비스

@refine — 간단한 프롬프트를 구조화된 프롬프트로

사용자 입력



정제 > 검색 > 리랭킹 > 생성



향상된 프롬프트

01

배경

왜 메타프롬프팅인가

문제 짧고 모호한 프롬프트 → 빗나간 답 → 다시 묻는 비효율. 토큰·시간 낭비.

핵심 인사이트 답은 “더 비싼 모델”이 아니라 “더 구조화된 프롬프트”.
소형 모델도 잘 설계된 프롬프트면 고품질 → 비용 절감.

실제 사용사례 원본 한 줄이 실무 설계 명세로 정제됩니다

BEFORE · 원본

“레이트 리미터 만들어줘”

@refine 정제 ▼

AFTER · 정제본

역할	대규모 트래픽 백엔드·분산 시스템 아키텍트 — 동시성·부하 분산·안정성 전문
임무	프로덕션급 Rate Limiter 설계·구현 — 단순 코드를 넘어 견고한 아키텍처 제안
필수 포함	① 알고리즘 비교 — Token Bucket · Sliding Window · Fixed Window (버스트·메모리 트레이드오프) ② 동시성 안전 구현 — Python/Go, Redis 분산으로 단일 인스턴스 한계 극복. 한국어 주석 첨부 ③ Race Condition 방지 — Lock 메커니즘 · Atomic 연산 ④ 아키텍처 — 배치 위치(Gateway vs App) · 파라미터(window-max) · 예외(HTTP 상태코드)
출력 형식	알고리즘 선택 근거 · 아키텍처 다이어그램 · 구현 코드 블록 · 동시성·확장성 요약

전체 구조 — FE 확장 ↔ BE 파이프라인

FRONTEND — VS Code 확장

@refine 참가자

개선된 프롬프트 체크 + 5선택지 제시

사이드바 (Webview)

원본→정제본 카드 · 검색

로그 (JSONL)

observe-YYYY-MM-DD.log

순수 JS · 런타임 의존성 0 · 승인 게이트 기반 UX

▼ HTTP POST /refine

BACKEND — FastAPI · 프롬프트 향상 매커니즘 4단계

1

LLM 정제

/rewrite

모호한 입력을 명료한 영어 프롬프트로 재작성. 의도 보존.

2

벡터 검색

/search

검증된 프롬프트 템플릿을 임베딩화하여 벡터DB 구성 -> 유저 입력을 임베딩해 코사인 유사도 Top-20 검색.

3

리랭킹

/rerank

Reranker로 의미 적합성 재점수화 → 최종 Top-3 선별.

4

LLM 최종 생성

/generate

Top-3 참고 및 원본 의도로 최종 프롬프트 작성.

@refine UX — 5가지 승인 선택지

✓ 전송

정제본 자동 전송 (기록 남김)

✎ 수정

정제본을 입력창에 채워 편집

↺ 다시 정제

동일 원본으로 재정제

→ 원본 전송

정제 없이 원본 그대로 전송

✕ 취소

@refine 원본을 입력창에 복원